BayFlow

Multi-Tenant Auto Repair Shop Management Platform

HACKATHON PROBLEM STATEMENT & BUILD BRIEF

Customer booking portal | Shop POS | Role-based repair workflow
Bonus track: in-app calling and AI Front Desk

Version 1.1 - October 2026

Contents

#	Section
1	Hackathon at a glance
2	Problem statement
3	What you will build: BayFlow
4	Users and roles
5	The complete booking lifecycle
6	Worked example
7	Core features and deliverables
8	Bonus features
9	Submission deliverables and repo standards
10	Judging criteria
11	FAQ

1. Hackathon at a Glance

BayFlow is a multi-tenant platform for independent auto repair shops. Customers discover a shop online, book a service and track their vehicle. Shop staff run the entire repair job, from check-in to pick-up, in one role-based POS. You are asked to design and ship a working version of this product, with a clean architecture and a live deployment.

Item	Detail
Project	BayFlow - multi-tenant auto repair shop management system
Format	Build a full working product: customer web app + shop POS + backend API
Tech stack	Free choice. Pick what you can ship well and defend in the review.
AI tooling	Allowed and encouraged: Cursor, Claude Code, Antigravity, Copilot, etc. You remain responsible for the quality of what ships.
Must submit	Public GitHub repository and a live URL (plus README and .env.example in repo root)
Scope tiers	Core features (mandatory) and bonus features (optional, judged separately)

2. Problem Statement

Most independent garages in Pakistan and many other markets still run on paper job cards, WhatsApp chats, phone calls and memory. Nothing is wrong with the mechanics. The process around them is what breaks. Each hand-off between customer, front desk, technician, parts counter and quality check is a place where information gets lost.

How it is done today

Stage	Manual reality today	Resulting pain
Finding a shop	Word of mouth, Google Maps, phone calls	No way to compare shops or see availability

Stage	Manual reality today	Resulting pain
Booking	Phone call or walk-in; appointment noted in a diary or WhatsApp	Double bookings, no-shows, no record of what the customer actually reported
Inspection	Technician checks the car and tells the advisor verbally or on a paper slip	Findings and part needs are not recorded; estimates are guesses
Estimate approval	Advisor phones the customer; approval is verbal	Disputes over price ("I never agreed to that"), no audit trail
Parts	Parts person checks shelves by eye; orders by phone; stock registers are updated late or never	Cars wait for parts that were never ordered; stock counts are wrong
Repair hand-offs	Job card physically passed between people	Lost cards, unclear ownership, nobody knows the car's current status
Quality check	Often skipped or informal, done by the same person who did the repair	Repeat visits, comebacks, unhappy customers
Customer updates	Customer calls repeatedly to ask "is my car ready?"	Advisors spend the day on the phone; missed calls mean lost customers
Multiple branches	Each branch keeps separate notebooks	Owner has no visibility across locations

Core problems to solve

- **No visibility:** neither customer nor owner can see where a vehicle is in the process.
- **No accountability:** approvals, estimates and QC results are not recorded.
- **Inventory chaos:** parts availability is unknown at estimate time, causing delays.
- **Communication overload:** advisors lose time and customers on phone calls, and missed calls are never followed up.
- **No multi-shop control:** an owner with several branches cannot manage separate teams and records cleanly.

3. What You Will Build: BayFlow

BayFlow is a single platform with three surfaces that share one backend:

- **Customer portal:** public landing page listing all shops, a guided booking wizard, and a login area where the customer tracks bookings and approves estimates.
- **Shop POS:** a role-based dashboard for Owner, Service Advisor, Technician, QC Inspector and Parts Person. Each role only sees the work that is theirs.
- **Backend API:** multi-tenant, authenticated, with a booking state machine that enforces who can move a job to which step.

How BayFlow solves the problems

Problem	BayFlow answer
Finding and booking	Public shop directory, service picker, real time slot availability, instant booking
Lost information	Customer's reported problem, vehicle details and every step are stored against one booking

Problem	BayFlow answer
Disputed estimates	Itemized estimate with parts and labour, sent to the customer, accepted in-app with timestamp
Wrong stock	Live inventory per shop; missing parts flagged; receiving parts updates stock automatically
Unclear ownership	Booking state machine: every status has exactly one responsible role
Skipped quality checks	Mandatory QC stage; failed QC returns the job to the technician with a recorded issue
Customer anxiety	Customer dashboard shows status and pending actions; notifications at key steps
Multi-branch chaos	Tenant isolation: each shop has its own team, inventory, slots and bookings; one owner can run many shops

What multi-tenant means here

A tenant is a shop. Every shop-owned record (team memberships, services, slots, inventory, bookings, estimates) carries a shop identifier, and every API query is scoped by it. A staff member of Shop A must never be able to read or change data of Shop B, even by guessing IDs. An owner can own several shops and each shop has its own separate team. Customers are global accounts and can book at any shop.

4. Users and Roles

Role	Who they are	Main responsibilities
Customer	Vehicle owner	Browse shops, book, log in, view bookings, approve or reject estimates, mark pick-up
Owner	Shop owner	Registers an account, creates shops, adds team members with roles, creates more shops later; full visibility of their shops
Service Advisor (SA)	Front desk	Confirms bookings, assigns technicians, reviews estimates, sends them to customer, assigns parts person, notifies customer when ready, closes booking
Technician	Mechanic	Sees assigned jobs, inspects the vehicle, adds required parts and cost estimate, performs repair, sends to QC
Parts Person	Inventory and procurement	Views stock, orders missing parts, confirms receipt (stock increases), allocates parts to a booking
QC Inspector	Quality control	Any QC person can pick a job from the QC queue, test it, then pass it or raise an issue

Access rules

- Roles are scoped per shop. A user can hold a role in one shop and a different role in another.
- An Owner can do everything inside their own shops. Shop staff accounts are created by the Owner, never by self sign-up.
- Technician sees only bookings assigned to them. QC sees the shared QC queue. SA and Owner see all bookings of the shop.
- The customer only sees their own bookings.

5. The Complete Booking Lifecycle

This is the heart of the product. The booking moves through a fixed set of statuses. Each status has one responsible role and a limited set of allowed next actions. Implement this as an explicit state machine in the backend and reject any invalid transition.

5.1 Customer-side booking journey

- Open the landing page and browse all listed shops.
- Select a shop and start a booking.
- Choose the problem or service: oil change, brake failure, check engine light, tire rotation, etc. (multiple selections allowed, plus free-text notes).
- See that shop's available time slots and pick one.
- Enter personal details (name, email, password) and vehicle details (registration number, make, model, year, optional color/mileage).
- Review and confirm. The booking is created with status PENDING and the customer account is created (or the existing account is reused when the email already exists and the password matches / the user is logged in).

5.2 Shop-side workflow

#	Status	Owner of step	What happens	Next
1	PENDING	Service Advisor	Booking appears on SA dashboard (with notification). SA reviews and confirms or declines.	CONFIRMED / CANCELLED
2	CONFIRMED	Service Advisor	SA assigns a technician.	ASSIGNED
3	ASSIGNED	Technician	Technician sees it on their dashboard and inspects the vehicle physically.	INSPECTING
4	INSPECTING	Technician	Technician adds needed parts, labour and total cost estimate, then submits to SA.	ESTIMATE_REVIEW
5	ESTIMATE_REVIEW	Service Advisor	SA reviews and edits if needed, then sends to customer.	AWAITING_CUSTOMER
6	AWAITING_CUSTOMER	Customer	Customer logs in, sees the pending estimate, accepts or rejects.	ESTIMATE_APPROVED / ESTIMATE_REJECTED
7	ESTIMATE_APPROVED	Service Advisor	SA is notified and assigns a parts person.	PARTS_PENDING
8	PARTS_PENDING	Parts Person	Checks inventory against required parts; creates purchase orders for missing items.	PARTS_ORDERED or PARTS_READY
9	PARTS_ORDERED	Parts Person	When parts arrive, confirms receipt: inventory increases.	PARTS_READY
10	PARTS_READY	Parts Person	Allocates parts to this booking (inventory reserved/deducted).	IN_REPAIR
11	IN_REPAIR	Technician	Job returns to the same technician, who performs the work and sends it to QC.	QC_PENDING
12	QC_PENDING	Any QC person	Any QC inspector picks the job from the queue.	QC_IN_PROGRESS

#	Status	Owner of step	What happens	Next
13	QC_IN_PROGRESS	QC Inspector	Tests the vehicle. If all good: confirm. If problem: create an issue.	READY_FOR_PICKUP (pass) / IN_REPAIR (fail, with issue)
14	READY_FOR_PICKUP	Service Advisor	SA notifies the customer that the vehicle is ready.	COMPLETED
15	COMPLETED	Customer / SA	Customer picks up the vehicle and the booking is completed.	Final

5.3 Edge cases to handle

- **Estimate rejected:** SA is notified and can revise the estimate with the technician, or cancel the booking.
- **Parts already in stock:** skip ordering and go straight from PARTS_PENDING to PARTS_READY.
- **Partial parts delivery:** receiving only some items keeps the booking in PARTS_ORDERED.
- **QC failure loop:** a failed QC creates an Issue record and returns the job to the technician; it can repeat until QC passes.
- **Slot conflicts:** two customers must not be able to book the same slot beyond its capacity (use a transaction or unique constraint).
- **Cancellation:** allowed by customer or SA before IN_REPAIR; reserved parts must be released back to stock.

5.4 Notifications (minimum)

Event	Who is notified
New booking created	Service Advisors of that shop
Technician assigned	That technician
Estimate submitted by technician	Service Advisor
Estimate sent to customer	Customer
Customer accepted or rejected estimate	Service Advisor
Parts ordered needed / parts ready	Parts Person / Technician
Sent to QC / QC failed	QC queue / Technician
Vehicle ready for pickup	Customer

In-app notifications (bell icon with unread count) are enough for core. Email is a nice addition.

6. Worked Example

Scenario: Ahmed owns a 2016 Honda Civic and the check engine light has come on. He wants an appointment at **Lahore Auto Care** (a shop owned by Mrs. Fatima Raza).

Step	Actor	Action	Booking status
0	Fatima (Owner)	Signs up, creates the shop "Lahore Auto Care" and adds Bilal (SA), Imran (Technician), Sara (QC), Usman (Parts).	-

Step	Actor	Action	Booking status
1	Ahmed	Opens BayFlow, sees 6 shops, selects Lahore Auto Care.	-
2	Ahmed	Selects "Check engine light" and "Oil change"; notes "Light came on yesterday, car vibrates at idle."	-
3	Ahmed	Sees Friday slots, picks 10:00 AM. Enters name, email ahmed@mail.com, password, plate LEA-1234, Honda Civic 2016. Confirms.	PENDING
4	Bilal (SA)	Sees the new booking on his dashboard, confirms it and assigns Imran.	CONFIRMED then ASSIGNED
5	Imran (Tech)	On Friday inspects the car. Diagnoses a faulty ignition coil. Adds: ignition coil x1 (PKR 6,500), oil filter x1 (PKR 900), engine oil 4L (PKR 5,200), labour (PKR 4,000). Total PKR 16,600. Submits.	ESTIMATE_REVIEW
6	Bilal	Reviews, adjusts labour to PKR 3,500 (total 16,100) and sends to Ahmed.	AWAITING_CUSTOMER
7	Ahmed	Logs in with the same email and password, sees the pending estimate banner and accepts.	ESTIMATE_APPROVED
8	Bilal	Gets the notification, assigns Usman (parts).	PARTS_PENDING
9	Usman	Sees stock: oil and filter available, ignition coil missing. Creates a purchase order for 1 coil.	PARTS_ORDERED
10	Usman	Next day the coil arrives. He confirms receipt (stock +1), then allocates all 3 parts to the booking (stock deducted).	PARTS_READY then IN_REPAIR
11	Imran	Sees the job back on his dashboard, performs the repair, sends to QC.	QC_PENDING
12	Sara (QC)	Picks it up, road tests. Finds a minor oil leak at the filter. Creates an issue.	IN_REPAIR (QC failed)
13	Imran	Fixes the seal, resubmits to QC.	QC_PENDING
14	Sara	Re-tests: all fine. Confirms.	READY_FOR_PICKUP (to SA)
15	Bilal	Sends "Your vehicle is ready for pick-up" to Ahmed.	READY_FOR_PICKUP
16	Ahmed	Collects the car; booking is completed.	COMPLETED

Bonus path: at step 6, Bilal is on another call and Ahmed phones the shop asking for an update. The AI Front Desk answers, looks up Ahmed by phone or booking ID, tells him the estimate is waiting for his approval, and leaves Bilal a summary and a task: "Customer Ahmed asked about estimate; reminded to approve in the portal."

7. Core Features and Deliverables

Core features are mandatory. Each module lists what must be delivered and how it will be checked.

7.1 Authentication and Multi-Tenancy

- Owner sign up and login; staff login with credentials created by the Owner; customer login with the email and password given during booking.
- Secure password hashing, token or session based auth, role checks on every protected endpoint.

- Tenant isolation enforced server side (shop ID scoped queries or row-level policies).
- **Acceptance:** a staff token for Shop A gets 403/404 when requesting Shop B's bookings.

7.2 Owner: Shops and Team Management

- Create a shop (name, address, city, phone, working hours, slot duration/capacity, logo optional).
- Add team members to a shop with a role (SA, Technician, QC, Parts Person); edit or deactivate them.
- Create additional shops later; each shop has an independent team, services, slots and inventory.
- Shop switcher in the dashboard for owners with multiple shops.
- Configure the service catalog (name, description, estimated duration, base price optional).

7.3 Customer Portal

- **Landing page:** hero section, list or grid of all shops (name, city, rating placeholder, services offered), basic search or city filter.
- **Booking wizard:** shop > services/problems > time slot > personal and vehicle details > review and confirm.
- Slot picker shows only available slots for the chosen date, per shop.
- **Customer dashboard:** list of bookings with live status timeline, pending actions highlighted (for example "Estimate awaiting your approval"), estimate detail with itemized parts and labour, accept/reject buttons, ready-for-pickup message.
- Responsive design that works on mobile.

7.4 Shop POS Dashboards

Role	Required dashboard views and actions
Service Advisor	Bookings list with filters by status; confirm/decline; assign technician; review/edit estimate; send to customer; assign parts person; notify ready; complete booking; notification bell
Technician	My assigned jobs; booking and vehicle detail; add estimate line items (part, quantity, unit cost, labour); submit to SA; start repair; send to QC; view QC issues sent back
Parts Person	Inventory table (SKU, name, quantity, reorder level, cost); jobs awaiting parts with required vs available; create purchase orders; receive orders (stock increases); allocate parts to booking
QC Inspector	Shared QC queue; pick a job; checklist/notes; pass or create issue (title, description) which returns the job to the technician
Owner	Overview of each shop: bookings by status, team list, inventory summary; can open any role view for support

7.5 Booking Engine and State Machine

- Single source of truth for statuses and allowed transitions, enforced in the service layer.
- Every transition recorded in a booking history/audit log (who, when, from, to, note).
- Role guard per transition (for example only a Technician assigned to the booking can submit an estimate).
- Estimate versioning or at least a revision count when SA edits before sending.

7.6 Inventory and Procurement

- Per-shop parts catalog and quantity on hand.
- Required parts compared to stock automatically to show shortages.
- Purchase orders with items and statuses (ORDERED, PARTIALLY_RECEIVED, RECEIVED).
- Receiving increases inventory; allocation decreases it and links parts to the booking; cancellation returns reserved parts.

7.7 Seed Data

- A seed script creating at least 3 shops, a team for each role, a service catalog, sample inventory and available slots, so judges can test instantly.
- Demo credentials listed in the README (and shown on the live site login page or README only, not hard-coded in code logic).

7.8 Core Definition of Done

- A judge can complete the full flow in section 5 on the live URL, from customer booking to COMPLETED, using seeded accounts.
- Both the QC fail loop and the estimate-rejected path work.
- No role can perform another role's action.

8. Bonus Features

Bonus features are optional and scored separately. Do not start them until the core flow works end to end.

8.1 In-App Calling between Service Advisor and Customer

Allow the SA and the customer to call each other directly from their dashboards, without sharing phone numbers.

- Call button on the booking page for both parties (SA to customer and customer to shop's SA).
- Browser-based voice using WebRTC, or a provider SDK such as Twilio Voice, LiveKit, Agora, Daily or Jitsi embed.
- Incoming call popup with accept and decline; call state: ringing, connected, ended, missed.
- Call log stored against the booking (who, when, duration, outcome).
- A missed call must be recorded and, in the AI track, can trigger the AI Front Desk.

8.2 AI Front Desk

If a customer calls and no SA picks up within a set time (for example 20 seconds), an AI voice agent answers instead.

- Greets the customer, asks who they are and why they are calling, and identifies them by name, phone, email or booking reference.
- Fetches the customer's booking(s) from the database through tool calls and answers accurately (current status, appointment time, pending estimate, ready for pick-up).

- Handles simple requests: reschedule request, "what is my status", "how much is the estimate".
- Never invents data. If it cannot answer, it says so and promises a callback.
- After the call, generates a summary (intent, details, sentiment) and creates a Task for the SA, such as "Call back Ahmed about estimate", linked to the booking and the call log.
- SA dashboard section: AI call summaries and open tasks, with mark-as-done.
- Suggested building blocks: LLM with function calling (OpenAI, Gemini, Claude or Groq), speech-to-text and text-to-speech (or a realtime voice API / Vapi / Retell / LiveKit Agents), and guardrails that limit the agent to the caller's own data.

8.3 Other Optional Ideas

- Email or SMS notifications; PWA or push notifications.
- Owner analytics: revenue, average turnaround time, QC fail rate per shop.
- Invoice PDF generation and payment status.
- Vehicle service history per car number.
- Real-time dashboard updates using WebSockets or server-sent events.

Bonus item	Expected evidence
In-app calling	Two browsers calling each other on the live site; call log entry
AI Front Desk	Demo video or live demo of unanswered call, conversation, then summary and task in SA dashboard

9. Submission Deliverables and Repo Standards

9.1 What you must submit

Deliverable	Requirement
GitHub repository	Public repo with meaningful commit history (not a single commit), organized folders, and no committed secrets
Live URL	Deployed and reachable without local setup; seeded demo data and demo accounts for every role
README.md (repo root)	Project overview, tech stack, prerequisites, install steps, how to run locally, how to seed, how to run tests, demo credentials, deployment notes, architecture summary
.env.example (repo root)	Every environment variable the project needs, with placeholder values and a short comment on each. Real values must never be committed
Optional	Short demo video (3 to 5 minutes), architecture diagram, API docs (Swagger/OpenAPI or Postman collection)

9.2 README must include

- **Prerequisites:** exact runtime and tool versions (for example Node 20+, Python 3.11+, PostgreSQL 15+, Docker).
- **Install dependencies:** exact commands for each app (frontend, backend).
- **Environment setup:** copy .env.example to .env and explain each variable.

- **Database:** how to create, migrate and seed.
- **Run:** one command or a few clear commands to start everything; ideally also docker compose up.
- **Demo accounts:** email and password for Owner, SA, Technician, QC, Parts and Customer.
- **Project structure and design decisions** in a short section.

9.3 Example .env.example

```
# --- App ---
NODE_ENV=development
PORT=4000
CLIENT_URL=http://localhost:3000

# --- Database ---
DATABASE_URL=postgresql://user:password@localhost:5432/bayflow

# --- Auth ---
JWT_SECRET=change-me
JWT_EXPIRES_IN=7d

# --- Email (optional) ---
SMTP_HOST=
SMTP_USER=
SMTP_PASS=

# --- Bonus: calling / AI (leave blank if not used) ---
CALL_PROVIDER_API_KEY=
LLM_API_KEY=
STT_TTS_API_KEY=
```

9.4 Code quality expectations

- Clean, readable code with consistent formatting (linter and formatter configured).
- Input validation on all endpoints; consistent error responses; proper HTTP status codes.
- Reusable UI components; centralized API client; typed code (TypeScript or Python type hints) is preferred.
- Sensible commit messages and small commits; AI-generated code must be reviewed and understood by the team.

10. Judging Criteria

Criteria	Weight	What judges look for
Core flow completeness	35%	Full lifecycle works live, including QC loop and estimate approval
Architecture and code quality	20%	Clean structure, state machine, tenant isolation, validation
Product and UX	15%	Clear landing page and booking wizard; role dashboards that show only relevant work
Deployment and docs	10%	Live URL works; README and .env.example let a stranger run it in minutes
Multi-tenancy and security	10%	Owner multi-shop, per-shop teams, access control
Bonus features	10%	Working calling and/or AI Front Desk with demo

Tip: deploy early and keep deploying. Teams that deploy at the last hour usually discover environment problems too late.

11. FAQ

Which tech stack should we use?

Any. Choose what your team can build and explain confidently. Judges evaluate the result and the code quality, not the brand of the framework.

Can we use AI tools like Cursor, Claude Code or Antigravity?

Yes. You are encouraged to. You must understand and be able to explain all code, and the output must still meet the clean-code standards above.

Are bonus features required?

No. Core features are mandatory and judged first. Bonus features are scored separately and only count if the core flow works.

Do we need real payments?

No. Cost estimates and approval are enough. Payment integration is optional.

How does a customer log in?

With the email and password entered during booking. If that email already exists, reuse the account instead of creating a duplicate.

Can one person have multiple roles?

Yes, per shop, if you support it, but each role's view and permissions must still work properly. A simple approach is one role per shop membership.

Can the Owner also act as other roles?

Owners should at least be able to view and manage all bookings in their shops. Allowing them to perform other roles' actions for support is optional.

How do multiple shops work for one owner?

One owner account can create many shops. Each shop has its own team members, services, slots, inventory and bookings, with a shop switcher in the POS.

Who can pick a QC job?

Any QC person of that shop. Make sure two people cannot pick the same job at once (first to pick locks it).

What should the live site contain?

Seeded data and demo accounts for all roles so judges can test without setup.

Does the slot system need to be complex?

No. Per-shop working hours with fixed duration slots and a capacity is enough. Booked slots must not be double-booked.

What if we cannot finish everything?

Prioritize a working end-to-end flow over extra polish. A smaller, correct, deployed product beats a large broken one.

How will the AI Front Desk be tested?

Via a demo: a simulated unanswered call, the AI identifying the customer, answering from real booking data, and a summary plus task appearing in the SA dashboard.

What goes in the submission?

GitHub repo link, live URL, README and .env.example in the repo root, and optionally a demo video.